
Analyse et refonte de l'architecture du système DNN//^{*}T_{cs}

Système parallèle collectif de deep learning pour la
reconnaissance de visages et la détection d'objets en temps réel

Réalisé par : *Douma Safa , Mrayhi Siwar et Ghazouani Farah*

Encadré par : *Maher Helaoui*

Article étudié : M. Helaoui et S. Bahroun, *Parallel Collective Tiny Deep Learning System for Face and Object Reidentification in Uncontrolled Environments*, Research Square, 2025.

Table des matières

Introduction générale	5
1 Présentation du système étudié	6
1.1 Contexte et objectifs	6
1.2 Les services fournis	6
1.3 Principe de fonctionnement : parallèle, collectif et Tiny	7
1.4 Cas d’usage : lunettes intelligentes pour personnes malvoyantes	7
1.5 Fondements théoriques	7
1.5.1 Structure d’évaluation	7
1.5.2 DNN parallèle	7
1.5.3 DNN parallèle collectif	8
1.5.4 Nombre optimal de processeurs	8
1.5.5 Approche Tiny	8
1.6 Résultats annoncés	8
2 Analyse de l’architecture actuelle	9
2.1 Vue d’ensemble	9
2.2 Architecture logique	9
2.3 Flux d’exécution	10
2.4 Architecture matérielle recommandée	10
2.5 Architecture du prototype	10
2.6 Caractérisation du style architectural	12
2.6.1 Une architecture client-serveur à client lourd	12
2.6.2 Une extension à trois tiers	12
2.6.3 Le patron API Gateway	12
2.6.4 S’agit-il d’une architecture microservices ?	12
2.6.5 Synthèse	13
2.7 Points faibles identifiés	13
3 Exigences et choix du style architectural	14
3.1 Attributs de qualité attendus	14
3.2 Styles architecturaux candidats	14
3.3 Style retenu	15
4 Architecture proposée	16
4.1 Vue d’ensemble en couches	16
4.2 Le cœur cloud orienté événements	17
4.3 Architecture interne du dispositif	18
4.4 Scénario de traitement d’une image	19
4.5 Patrons d’architecture mobilisés	19
4.6 Sécurité et confidentialité	19
4.7 Choix technologiques	20

4.8	Justification : réponse aux exigences	20
4.9	Correspondance avec l'architecture initiale	21
4.10	Limites de la proposition	21
5	Modélisation selon le modèle C4	22
5.1	Présentation du modèle C4	22
5.2	Niveau 1 : diagramme de contexte	22
5.3	Niveau 2 : diagramme de conteneurs	23
5.4	Niveau 3 : diagramme de composants du service de fusion	25
5.5	Niveau 4 : code du moteur de fusion	27
5.6	Vue dynamique : traitement d'une image	27
	Conclusion générale	29
	Bibliographie	30

Table des figures

2.1	Architecture actuelle : clients, gateway REST, serveur multi-services et objets Tiny	9
2.2	Prototype : service de reconnaissance de visage découpé en processus	11
4.1	Architecture proposée : vue d'ensemble en couches	16
4.2	Cœur cloud : microservices d'inférence et fusion collective orientés événements . . .	17
4.3	Architecture interne du dispositif	18
5.1	C4 niveau 1 : diagramme de contexte du système	23
5.2	C4 niveau 2 : diagramme de conteneurs	24
5.3	C4 niveau 3 : composants du service de fusion collective	26
5.4	C4 niveau 4 : diagramme de classes du moteur de fusion	27

Liste des tableaux

1.1	Décomposition du nom du système	6
1.2	Synthèse des résultats expérimentaux de l'article	8
2.1	Composants du gestionnaire parallèle collectif	10
2.2	Organisation du prototype : un service par processeur	11
2.3	Confrontation du prototype aux critères d'une architecture microservices	12
2.4	Faiblesses de l'architecture actuelle et leurs conséquences	13
3.1	Exigences et attributs de qualité du système	14
3.2	Comparaison des styles architecturaux candidats	15
4.1	Patrons d'architecture et de conception utilisés	19
4.2	Technologies proposées par couche	20
4.3	Réponse de l'architecture proposée à chaque exigence	20
4.4	Correspondance entre l'architecture initiale et l'architecture proposée	21
5.1	Notation utilisée dans les diagrammes C4	22
5.2	Conteneurs du système et leurs responsabilités	25
5.3	Vue dynamique : traitement d'une image de bout en bout	28

Introduction générale

Les systèmes de vision par ordinateur embarqués (lunettes intelligentes pour personnes mal-voyantes, véhicules autonomes, vidéosurveillance intelligente) doivent concilier deux exigences contradictoires : réagir en temps réel sur un équipement aux ressources limitées, et atteindre une précision élevée, qui nécessite généralement des modèles de deep learning volumineux.

L'article de Helaoui et Bahroun [1] propose une réponse à ce compromis avec le système DNN//*T_{cs} (*Parallel Collective Tiny Deep Learning client-server System*). Ce système exécute en parallèle plusieurs versions de modèles de reconnaissance (visages, émotions, genre, objets) sur un client léger et sur un serveur puissant, combine leurs résultats de manière « collective » et s'appuie sur des objets connectés (approche « Tiny ») pour sélectionner la meilleure réponse.

Ce rapport a trois objectifs :

1. **comprendre et expliquer** ce que fait le système et sur quels fondements il repose ;
2. **analyser son architecture actuelle**, caractériser le style architectural réellement mis en œuvre et en identifier les faiblesses ;
3. **proposer une nouvelle architecture**, mieux adaptée aux exigences du système, justifier ce choix et la documenter selon le **modèle C4** [3].

Le chapitre 1 présente le système étudié. Le chapitre 2 analyse son architecture actuelle. Le chapitre 3 formalise les exigences et compare les styles architecturaux candidats. Le chapitre 4 décrit l'architecture proposée, et le chapitre 5 la modélise selon les niveaux du modèle C4.

Chapitre 1

Présentation du système étudié

1.1 Contexte et objectifs

Le système DNN//*T_{cs} vise à assister une machine autonome dans sa prise de décision à partir de ce qu'elle perçoit par une caméra. Le nom du système résume son principe :

Table 1.1 – Décomposition du nom du système

Élément	Signification
DNN	<i>Deep Neural Network</i> : réseau de neurones profond au sens large (CNN, RNN, etc.)
//	Parallèle : les calculs sont répartis sur plusieurs processeurs
*	Collectif : plusieurs versions de modèles coopèrent pour produire un résultat commun
T	<i>Tiny</i> : utilisation de microcontrôleurs (approche TinyML)
cs	<i>Client-server</i> : répartition des traitements entre un client et un serveur

Le problème traité est un compromis classique en vision embarquée. Un modèle léger (par exemple YOLOv8n) s'exécute rapidement sur un smartphone mais commet davantage d'erreurs ; un modèle lourd (YOLOv8x) est plus précis mais trop coûteux pour l'appareil. De plus, les auteurs constatent qu'aucun modèle n'est le meilleur dans toutes les conditions (éclairage, pose du visage, occlusions) : selon l'environnement, une méthode peut en dominer une autre.

1.2 Les services fournis

Le système fournit quatre services de perception :

- **reconnaissance de visages** : identifier la personne présente dans l'image ;
- **reconnaissance d'émotions** : reconnaître l'état émotionnel (joie, colère, peur, surprise, etc.) ;
- **détection du genre** ;
- **détection d'objets** : localiser et classer les objets de la scène (bateau, voiture, animal, etc.) à l'aide de YOLOv8.

1.3 Principe de fonctionnement : parallèle, collectif et Tiny

La solution proposée combine trois idées.

Le parallélisme. Il permet d'exécuter plusieurs modèles simultanément sans perdre la réactivité, mais aussi de paralléliser les calculs à l'intérieur d'un même réseau, en calculant les neurones d'une même couche en même temps.

Le collectif. Plutôt que de faire confiance à un seul modèle, le système exécute plusieurs versions (CNN, RNN, différentes tailles de YOLOv8, etc.) sur la même image et combine leurs résultats. Le client exécute des versions légères et produit une réponse locale ; le serveur exécute des versions plus précises et renvoie une **alerte** au client lorsque son résultat diffère.

Le Tiny. Chaque objet réel peut être équipé d'un microcontrôleur (une carte Arduino dans le prototype) qui transmet directement son identité, sa position et ses dimensions. Ces données servent de référence pour choisir, parmi les versions exécutées, celle dont le résultat est le plus fiable.

1.4 Cas d'usage : lunettes intelligentes pour personnes malvoyantes

L'article illustre le système par des lunettes permettant à une personne malvoyante de bénéficier, en temps réel, des informations accessibles à une personne voyante. Le problème global P est décomposé en trois sous-problèmes exécutés en parallèle :

- P_1 : reconnaître et classer les objets (auquel s'ajoutent en parallèle la reconnaissance de visages, du genre et des émotions) ;
- P_2 : éviter les objets dynamiques, en estimant leur trajectoire ;
- P_3 : éviter les obstacles statiques, grâce à deux capteurs de position.

Les auteurs indiquent que ce cas se généralise à la vidéosurveillance intelligente et aux véhicules autonomes.

1.5 Fondements théoriques

1.5.1 Structure d'évaluation

Pour combiner les résultats de plusieurs modèles, les auteurs utilisent une structure d'évaluation $S = (E, \leq, \otimes)$: un ensemble ordonné de valeurs E , un opérateur de comparaison $\leq$ et un opérateur de combinaison $\otimes$ commutatif, associatif et monotone :

$$\alpha \leq \beta \Rightarrow \alpha \otimes \lambda \leq \beta \otimes \lambda, \quad \forall \alpha, \beta, \lambda \in E. \quad (1.1)$$

1.5.2 DNN parallèle

En reprenant les travaux de Lee et al. [2], les auteurs remarquent que les couches d'un réseau sont dépendantes (une couche utilise la sortie de la précédente) alors que les neurones d'une même couche sont indépendants. D'après le théorème de Brent et la loi d'Amdahl, le nombre optimal de processeurs pour accélérer un réseau est donc au moins égal au nombre de neurones par couche $|N^l|$ (lemme 1). Le temps de calcul d'une couche se réduit alors à celui d'un seul neurone, augmenté du coût de communication.

1.5.3 DNN parallèle collectif

Un algorithme « parallèle collectif » est défini comme la combinaison des résultats de plusieurs versions parallèles :

$$\text{DNN} //^* = \bigotimes \text{DNN} //, \quad \text{DNN} //_{cs}^* = \left(\bigotimes \text{DNN} //_c \right) \otimes \left(\bigotimes \text{DNN} //_s \right). \quad (1.2)$$

1.5.4 Nombre optimal de processeurs

Côté client, pour N_O objets et N_F visages dans une image, avec α_c analyses par objet et β_c analyses par visage (théorème 3) :

$$P_c^* = (\alpha_c N_O + \beta_c N_F) \times |N_c^l|. \quad (1.3)$$

Par exemple, avec une analyse par objet (détection), trois analyses par visage (identité, genre, émotion), deux objets, un visage et 64 neurones par couche : $P_c^* = (2 + 3) \times 64 = 320$ processeurs.

Côté serveur, avec k_1 versions de DNN et k_2 algorithmes d'apprentissage supplémentaires (théorème 4) :

$$P_s^* = k_1(\alpha_s N_O + \beta_s N_F) \times |N_s^l| + k_2. \quad (1.4)$$

1.5.5 Approche Tiny

Les théorèmes 1 et 2 affirment que si chaque objet (ou chaque personne) est connecté et transmet lui-même ses caractéristiques (classe ou identité, position, dimensions), alors la détection d'objets et l'identification de visages se réduisent à de petits programmes exécutables sur microcontrôleur.

1.6 Résultats annoncés

Table 1.2 – Synthèse des résultats expérimentaux de l'article

Jeu de données	Versions classiques	Version Tiny
Multi-PIE (visages)	Client seul : 87 à 94 % ; versions serveur : 97 à 99,7 %	100 % (identité transmise par une application mobile)
Bosphorus (émotions)	86 à 100 % selon l'émotion	100 % (émotion transmise par l'utilisateur)
YouTube-BoundingBoxes étendu (objets)	Nombreuses fausses détections de YOLOv8n et YOLOv8m	Aucune fausse détection (objets supposés connectés)

Ces résultats doivent être interprétés avec prudence : les scores de 100 % découlent directement de l'hypothèse selon laquelle l'identité réelle est transmise par l'objet ou la personne elle-même.

Chapitre 2

Analyse de l'architecture actuelle

2.1 Vue d'ensemble

L'architecture décrite dans l'article s'organise autour de trois types d'acteurs : des clients (dispositifs mobiles), un serveur puissant et des objets connectés dits « Tiny ». La figure 2.1 en donne une vue synthétique, reconstituée à partir des figures 1, 2 et 11 de l'article.

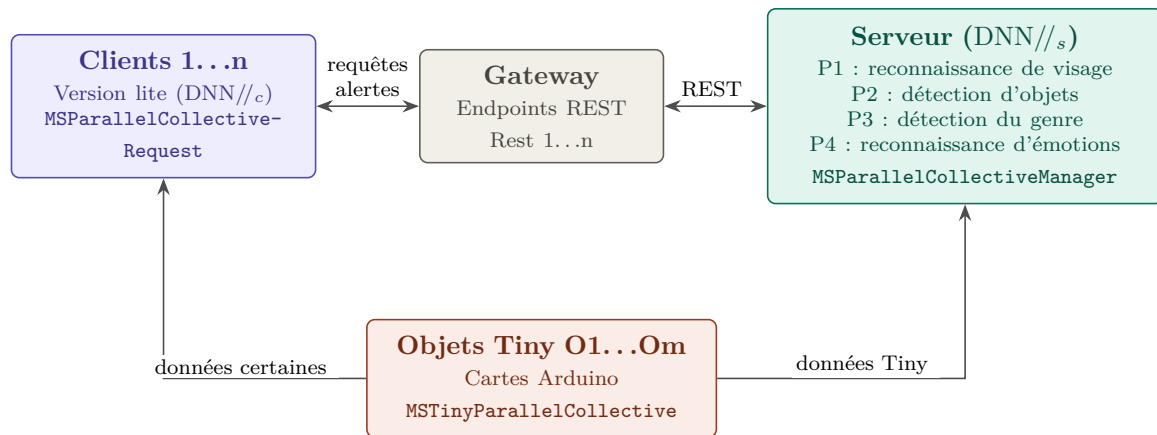


Figure 2.1 – Architecture actuelle : clients, gateway REST, serveur multi-services et objets Tiny

2.2 Architecture logique

Version client-serveur simple (figure 1 de l'article). Plusieurs clients accèdent, via une gateway et des endpoints REST, à un serveur qui héberge les quatre services. Chaque service est affecté à un processeur : le processeur 1 pour la reconnaissance de visage, le 2 pour la détection d'objets, le 3 pour le genre et le 4 pour les émotions.

Version Tiny (figure 2 de l'article). Un troisième niveau est ajouté : les « Tiny Services » (objets O1..Om). Chaque client embarque le micro-service `MSParallelCollectiveRequest` et reçoit une éventuelle correction à la fois du serveur et des objets connectés.

Gestionnaire collectif (figure 11 de l'article). La coordination repose sur trois composants, décrits dans le tableau 2.1.

Table 2.1 – Composants du gestionnaire parallèle collectif

Composant	Emplacement	Rôle
MSParallelCollective-Request (algo. 13)	Client	Envoie une requête à chaque service, reçoit directement certaines données des objets Tiny puis le résultat collectif final.
MSTinyParallelCollective (algo. 14)	Microcontrôleur	Envoie les données de l'objet au client (si demandé) et systématiquement au serveur.
MSParallelCollective-Manager (algo. 15)	Serveur	Compare les versions d'un même service, choisit celle de meilleure confiance (calculée avec les données Tiny) et n'envoie que ce résultat au client.

2.3 Flux d'exécution

Côté client (algorithme 1 de l'article).

1. `Collect()` capture l'image ou la vidéo ;
2. `Share(i, d)` l'envoie au serveur avec des requêtes (version plus précise du DNN, k_1 autres versions, k_2 algorithmes d'apprentissage) ;
3. le client exécute localement, en parallèle, un DNN// c par objet et par analyse, puis par visage et par analyse ;
4. `SendResult(cr)` transmet son résultat combiné au serveur ;
5. `Receive()` récupère une éventuelle alerte ;
6. `ManagerClient(cr, alert)` détermine le résultat présenté à l'utilisateur.

Côté serveur (algorithme 2 de l'article).

1. `ReceiveShared()` reçoit l'image et les requêtes ;
2. `ParallelRun()` exécute en parallèle k_1 versions (CNN, DNN, RNN...) et k_2 algorithmes d'apprentissage, chacune parallélisée par objet et par visage ;
3. `ReceiveResult()` récupère le résultat calculé par le client ;
4. `ManagerServer()` compare les deux résultats et produit une alerte en cas de différence ;
5. `Send(alert)` renvoie la correction au client.

2.4 Architecture matérielle recommandée

Sur le dispositif, les auteurs proposent un système multi-agents d'environnement 70 agents, idéalement un agent par cœur : de 8 à 64 cœurs pour P_1 (un par neurone du modèle léger, soit idéalement huit processeurs octa-cœurs à mémoire distribuée), au moins un cœur pour P_2 et au moins un cœur pour P_3 . Sur le serveur, la même logique est appliquée à plus grande échelle : au moins 512 cœurs pour le DNN principal, plus $512 \times k$ cœurs pour les k autres versions.

2.5 Architecture du prototype

Le prototype a été réalisé sur un ordinateur portable (processeur Intel Core i3 à 4 cœurs, 12 Go de mémoire, Ubuntu 22.04), en Python avec les bibliothèques `multiprocessing`, `OpenCV`,

Ultralytics YOLOv8, Keras et pytsx3. Chaque cœur héberge un service, lui-même découpé en « micro-services » qui sont en réalité des processus.

Table 2.2 – Organisation du prototype : un service par processeur

Processeur	Service	Processus (« micro-services »)
1	Reconnaissance de visage (LBPH, puis CNN)	P1 MSLBPHFaceRecognition, P2 MSShowResult, P3 MSFaceDynamicDataSet, P5 MSFaceDynamicTraining
2	Détection d'objets (YOLOv8n)	P1 MSLunchPredict, P2 MSInit, P3 MSColor, P4 MSConfig, P5 MSStartPredictor, P6 MSSayResult, P7 MSDetectionShowResult, P8 MTrackerShowResult
3	Détection du genre (DNN, Keras)	P1 MSDetectGender, P2 MSDNNDetectFaces, P3 MSSayResult, P4 MSShowResult
4	Reconnaissance d'émotions (DNN, Keras)	P1 MSEmotionRecognition, P2 MSDNNEmotionRecognition, P3 MSSayResult, P4 MSShowResult

La figure 2.2 détaille le service de reconnaissance de visage. Lorsque la confiance dépasse 60 %, le processus 4 capture 30 nouvelles images du visage, puis le processus 5 réentraîne le modèle et le service est redémarré. Grâce à ce jeu de données dynamique, la confiance passe de 30–45 % à plus de 60 % ; une seconde version fondée sur un CNN atteint 96 %.

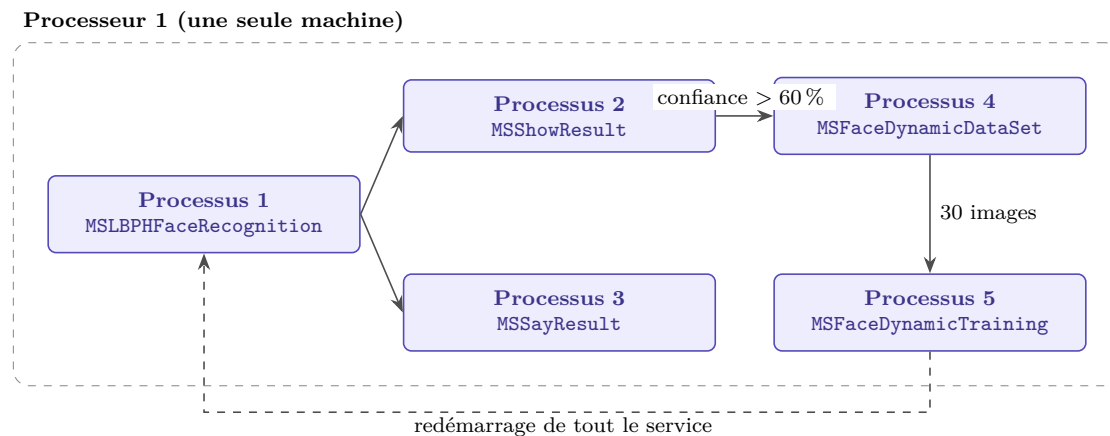


Figure 2.2 – Prototype : service de reconnaissance de visage découpé en processus

Les services de genre et d'émotion suivent un schéma en boucle : le processus 1 lit une image, le processus 2 exécute le réseau, les processus 3 et 4 affichent et prononcent le résultat, puis le processus 1 est appelé.

2.6 Caractérisation du style architectural

2.6.1 Une architecture client-serveur à client lourd

Au niveau global, il s'agit bien d'une architecture client-serveur, comme l'indique le nom du système. Il ne s'agit toutefois pas d'un client léger : le client exécute lui-même des modèles et produit un résultat local. Le serveur ne travaille pas à sa place ; il refait le calcul avec des modèles plus lourds et joue un rôle de **correcteur**. Le schéma est donc de type « calcul local et vérification distante ».

2.6.2 Une extension à trois tiers

L'ajout des objets Tiny introduit un troisième niveau qui communique à la fois avec le client et le serveur. Dans le vocabulaire actuel, cette organisation s'apparente à une architecture *edge/cloud* associée à l'Internet des objets [11] : objets connectés (terrain), dispositif mobile (edge) et serveur (cloud), même si les auteurs n'emploient pas ces termes.

2.6.3 Le patron API Gateway

Les clients accèdent au serveur par une passerelle unique suivie d'endpoints REST, ce qui correspond au patron *API Gateway*, caractéristique des architectures orientées services.

2.6.4 S'agit-il d'une architecture microservices ?

L'article qualifie ses composants de micro-services (préfixe « MS »). Le tableau 2.3 les confronte aux critères usuels de ce style [4, 5].

Table 2.3 – Confrontation du prototype aux critères d'une architecture microservices

Critère	Situation dans le système actuel	Respecté
Déploiement indépendant	Tous les composants appartiennent au même programme et sont lancés ensemble	Non
Communication réseau légère	Appels directs de processus (<code>RunProcess2(...)</code>), sans réseau	Non
Organisation par capacité métier	Oui pour visage, objets, genre et émotion ; non pour <code>MSShowResult</code> ou <code>MSSayResult</code> , simples fonctions techniques	Partiel
Données propres à chaque service	Le dossier du jeu de données de visages est partagé entre plusieurs composants	Non
Mise à l'échelle indépendante	Chaque service est lié à un processeur fixe	Non
Isolation des pannes	Chaînes d'appels récursives (<code>MSDetectGender</code> → <code>MSDNNDetectFaces</code> → <code>MSDetectGender</code>) : un maillon défaillant arrête la boucle	Non

Le cas du réentraînement est révélateur : après `MSFaceDynamicTraining`, tout le service est redémarré, alors qu'une architecture microservices ne redéploierait que le service concerné. **L'intention est donc orientée services, mais l'implémentation correspond à un monolithe modulaire multi-processus** : une seule application, découpée en modules exécutés dans des processus distincts sur une même machine.

2.6.5 Synthèse

L'architecture actuelle peut être caractérisée ainsi : *une architecture client-serveur à client lourd, étendue à trois tiers par des objets connectés, dont l'accès au serveur passe par une API Gateway et des services REST, et dont chaque service est implémenté comme un ensemble de processus d'une même application (monolithe modulaire multi-processus).*

2.7 Points faibles identifiés

Table 2.4 – Faiblesses de l'architecture actuelle et leurs conséquences

Faiblesse	Conséquence
Faux micro-services (processus d'un même programme)	Impossible de déployer, mettre à jour ou mettre à l'échelle un service seul
Couplage rigide entre matériel et logiciel (« un service par processeur », « un processeur par neurone »)	Architecture non portable ; en pratique, la parallélisation des neurones est assurée par les GPU et accélérateurs, pas par des cœurs CPU dédiés
Communication synchrone par REST	Mal adaptée à un flux vidéo continu ; latence accrue par l'attente des réponses
Gestionnaire collectif centralisé	Point de défaillance unique, pas de mode dégradé ni de tolérance aux pannes
Fusion par « meilleure confiance »	Les scores de modèles différents ne sont pas calibrés, donc pas directement comparables
Hypothèse Tiny très forte (tout objet est connecté)	Scores de 100 % quasi tautologiques ; hypothèse irréaliste dans la plupart des environnements, comme le reconnaissent les auteurs
Absence de mesures de latence	L'exigence de temps réel, argument central, n'est pas démontrée
Absence de sécurité et de protection des données	Des données biométriques (visages) circulent et sont stockées sans protection décrite
Réentraînement avec redémarrage	Interruption du service pendant la mise à jour du modèle

Chapitre 3

Exigences et choix du style architectural

3.1 Attributs de qualité attendus

Le choix d’une architecture doit partir des exigences et des attributs de qualité [6], et non des technologies. Le tableau 3.1 les recense.

Table 3.1 – Exigences et attributs de qualité du système

N°	Exigence	Description	Priorité
E1	Temps réel	Une personne malvoyante ou un véhicule ne peut pas attendre une réponse distante pour réagir	Haute
E2	Traitement de flux	Les données sont des flux vidéo continus, pas des requêtes ponctuelles	Haute
E3	Exécution collective	Plusieurs versions d’un même modèle traitent la même image et leurs résultats sont combinés	Haute
E4	Extensibilité	Ajouter un nouveau modèle ou une nouvelle version sans modifier le reste du système	Haute
E5	Hétérogénéité	Microcontrôleurs, smartphones et serveurs GPU coexistent	Moyenne
E6	Disponibilité	Le dispositif doit continuer à fonctionner si le réseau est coupé	Haute
E7	Confidentialité	Les visages sont des données biométriques sensibles (RGPD)	Haute
E8	Scalabilité	Nombreux clients simultanés et charge variable	Moyenne
E9	Observabilité	Mesurer la latence et la précision de chaque version	Moyenne

3.2 Styles architecturaux candidats

Quatre options ont été étudiées.

Monolithe embarqué. Tout est exécuté sur le dispositif. Cette option offre le meilleur temps de réponse et fonctionne hors ligne, mais les ressources limitées empêchent d’exécuter plusieurs versions lourdes : l’exigence collective (E3) est sacrifiée.

Client-serveur REST (architecture actuelle). Le principe de correction par le serveur est pertinent, mais le couplage fort, la communication synchrone et l'absence d'isolation limitent la réactivité, l'extensibilité et la disponibilité.

Microservices entièrement dans le cloud. Cette option offre une excellente extensibilité et scalabilité, mais toute décision dépend du réseau : elle est incompatible avec le temps réel (E1) et le fonctionnement hors ligne (E6).

Architecture hybride Edge–Cloud orientée événements. La décision immédiate est prise sur le dispositif, tandis que le cloud, organisé en microservices communiquant par événements, exécute les versions lourdes et renvoie des corrections de manière asynchrone.

Table 3.2 – Comparaison des styles architecturaux candidats

Style	Temps réel	Collectif et extensi- bilité	Hors ligne	Scalabilité	Verdict
Monolithe embarqué	Bon	Très faible	Oui	Aucune	Rejeté
Client-serveur REST (actuel)	Moyen	Faible	Partiel	Limitée	À améliorer
Microservices 100 % cloud	Mauvais	Très bon	Non	Très bonne	Rejeté
Edge–Cloud orienté événements	Bon	Très bon	Oui	Très bonne	Retenu

3.3 Style retenu

Le style retenu est une **architecture hybride Edge–Fog–Cloud, orientée événements, avec de véritables microservices côté cloud**. Elle conserve les idées fondatrices de l'article (client léger, serveur correcteur, exécution collective, apport des objets connectés) tout en les implémentant avec des patrons éprouvés.

Chapitre 4

Architecture proposée

4.1 Vue d'ensemble en couches

Le système est organisé en couches selon la distance au terrain : plus on s'éloigne de l'utilisateur, plus la puissance de calcul augmente, mais plus la latence augmente également (figure 4.1).

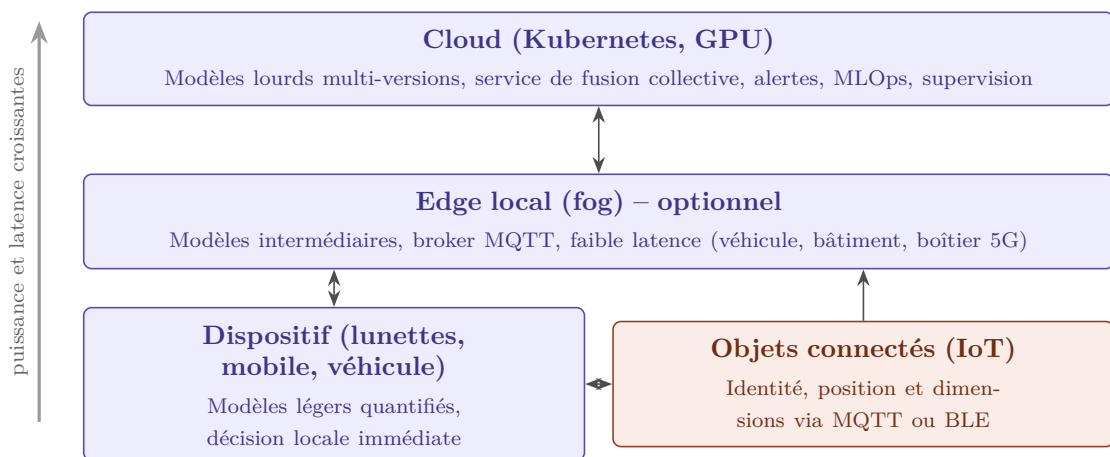


Figure 4.1 – Architecture proposée : vue d'ensemble en couches

Le dispositif (edge device). Il exécute des modèles légers et quantifiés (par exemple YOLOv8n et un petit CNN de visage) et **prend lui-même la décision immédiate**. Il garantit ainsi le temps réel et le fonctionnement hors ligne.

L'edge local (fog) [12]. Petit serveur proche de l'utilisateur (embarqué dans le véhicule, installé dans le bâtiment surveillé ou relié en Wi-Fi/5G), il exécute des modèles intermédiaires avec une latence de quelques millisecondes et héberge le broker MQTT qui collecte les données des objets connectés. Cette couche est optionnelle : pour des lunettes utilisées en extérieur, le dispositif peut communiquer directement avec le cloud.

Le cloud. Il exécute les modèles lourds (YOLOv8l et YOLOv8x, plusieurs versions de CNN), réalise la fusion collective et gère le cycle de vie des modèles.

Les objets connectés. Ils publient leur identité et leur position. Ils deviennent une **source d'information supplémentaire** dans la fusion et non plus une condition indispensable, ce qui corrige l'hypothèse irréaliste de l'architecture initiale.

4.2 Le cœur cloud orienté événements

C'est dans le cloud que l'idée de « parallèle collectif » trouve son implémentation naturelle (figure 4.2).

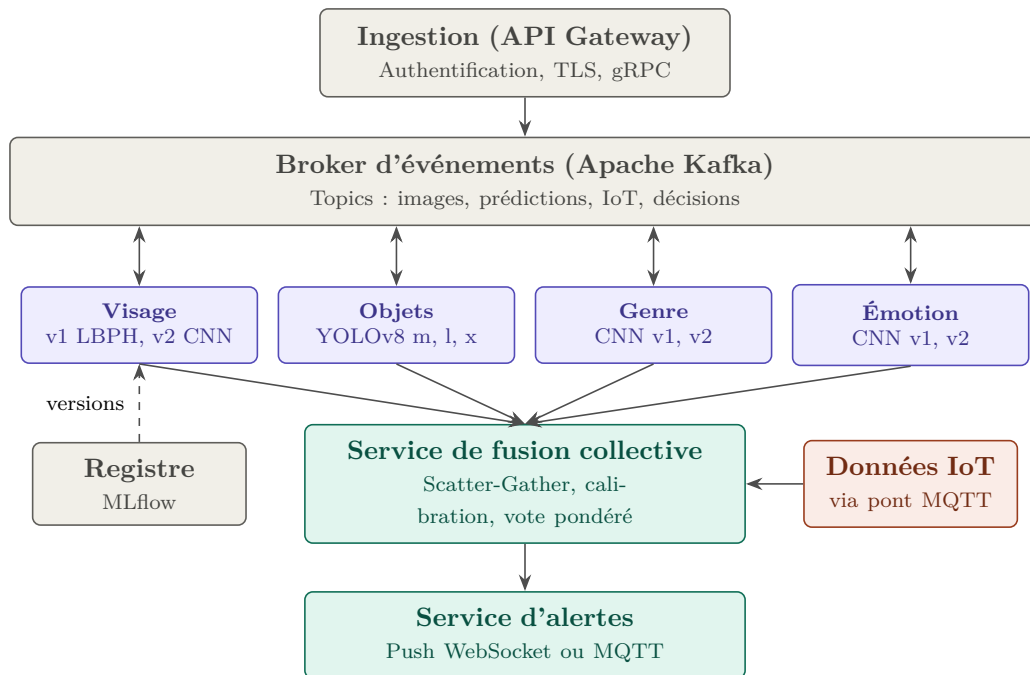


Figure 4.2 – Cœur cloud : microservices d'inférence et fusion collective orientés événements

Ingestion (API Gateway). Point d'entrée unique, elle gère l'authentification, le chiffrement TLS et la limitation de débit. Elle ne traite pas l'image mais la publie dans le broker. Pour la vidéo, gRPC en streaming ou MQTT remplacent REST, qui impose d'ouvrir une requête et d'attendre la réponse pour chaque image.

Broker d'événements. Apache Kafka [8] remplace les appels directs entre services. Chaque image devient un événement publié sur un *topic* ; tous les services abonnés le reçoivent simultanément. Ce patron **Publish/Subscribe** [7] réalise exactement l'exécution parallèle collective de l'article : une même image est traitée en même temps par toutes les versions, sans qu'aucun service ne connaisse les autres.

Services d'inférence. Un microservice par tâche ; chaque version de modèle est déployée comme une instance séparée, dans son propre conteneur, avec un serveur d'inférence de type Triton ou TorchServe sur GPU. Ajouter une nouvelle version (YOLOv9 par exemple) revient à déployer un conteneur supplémentaire abonné au même topic, sans modifier le reste du système : c'est le principe ouvert/fermé appliqué à l'architecture.

Service de fusion collective. Il remplace `MSParallelCollectiveManager` en l'améliorant. Il regroupe les réponses des différentes versions pour une même image (patron **Scatter-Gather**) avec un délai maximal : une version trop lente est ignorée. Les scores sont d'abord **calibrés** [10] pour être comparables, puis combinés par un **vote pondéré** dans lequel les données IoT, lorsqu'elles existent, constituent une preuve de poids élevé (algorithme 1).

Service d'alertes. Il compare le résultat fusionné à la décision déjà prise par le dispositif et ne pousse une correction **qu'en cas de désaccord**, ce qui économise la bande passante.

Services transverses. Le registre de modèles et le pipeline MLOps (MLflow) remplacent le réentraînement avec redémarrage : les modèles sont réentraînés en arrière-plan puis déployés progressivement, sans interruption. La supervision (Prometheus, Grafana) mesure la latence et la précision de chaque version, ce qui permet de démontrer l'exigence de temps réel.

Algorithme 1 Fusion collective d'une image (Scatter-Gather et vote pondéré)

Entrées : image f , ensemble V des versions actives, délai maximal Δ , poids w_v de chaque version, poids λ de la preuve IoT

Sortie : décision fusionnée D

```

1:  $R \leftarrow \emptyset$ 
2: tant que  $|R| < |V|$  et temps écoulé  $< \Delta$  faire
3:    $r \leftarrow \text{ATTENDREPREDICTION}(f)$ 
4:    $R \leftarrow R \cup \{\text{CALIBRER}(r)\}$ 
5: fin tant que
6:  $I \leftarrow \text{DONNEESIoT}(f)$  ▷ peut être vide
7: pour tout étiquette candidate  $\ell$  faire
8:    $s(\ell) \leftarrow \sum_{r \in R, r.\text{label}=\ell} w_{r.\text{version}} \cdot r.\text{score} + \lambda \cdot [\ell \in I]$ 
9: fin pour
10:  $D \leftarrow \arg \max_{\ell} s(\ell)$ 
11: si  $D \neq$  décision locale du dispositif alors
12:    $\text{PUBLIERALERTE}(f, D)$ 
13: fin si
14: retourner  $D$ 

```

4.3 Architecture interne du dispositif

La règle fondamentale est la suivante : **le dispositif ne dépend jamais du cloud pour agir** (figure 4.3).

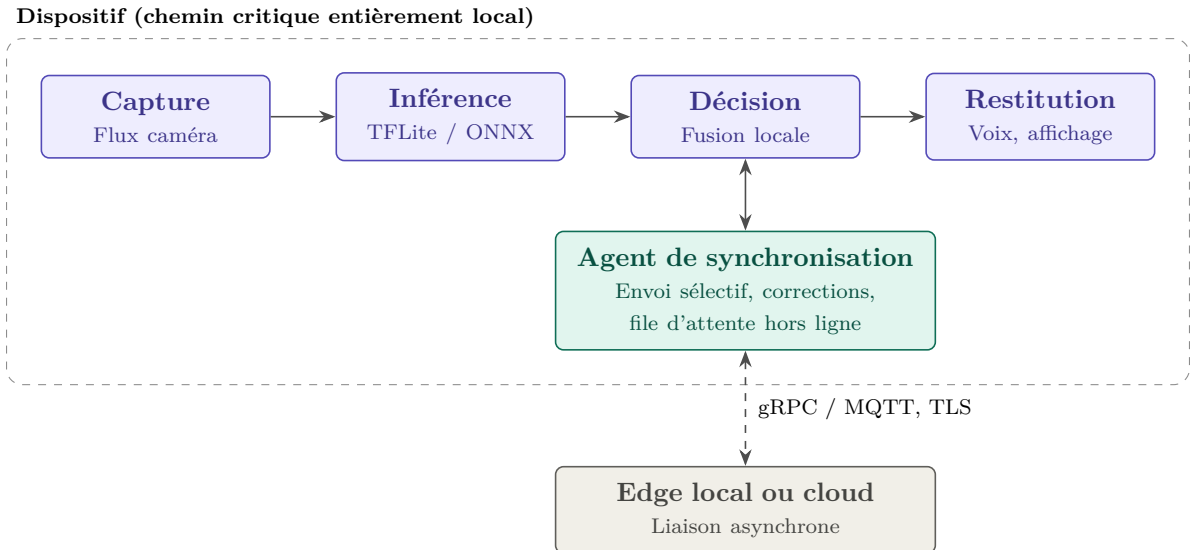


Figure 4.3 – Architecture interne du dispositif

Le chemin principal (capture, inférence, décision, restitution) est entièrement local. Les modèles sont convertis en TensorFlow Lite ou ONNX et quantifiés (par exemple en entiers sur 8 bits) pour s'exécuter sur le NPU ou le GPU du smartphone ; l'utilisateur est prévenu en quelques dizaines de millisecondes.

L’agent de synchronisation fonctionne de manière asynchrone. Il n’envoie que les données utiles : une image par seconde plutôt que trente, ou les images pour lesquelles la confiance locale est faible. Lorsqu’une correction arrive, la décision est mise à jour. En cas de coupure réseau, le dispositif passe en **mode dégradé** : il continue avec ses modèles locaux et met les événements en attente. Les appels distants sont protégés par un **disjoncteur** (*Circuit Breaker*) [13].

4.4 Scénario de traitement d’une image

1. **Vers 30 ms** : le dispositif capture l’image, exécute ses modèles légers et annonce le résultat à l’utilisateur.
2. **En parallèle** : l’agent de synchronisation publie l’image (ou ses embeddings) vers le cloud.
3. **Vers 150 à 300 ms** : toutes les versions du cloud ont traité l’image et le service de fusion a produit la décision collective.
4. **Uniquement en cas de désaccord** : le service d’alertes pousse une correction, qui est annoncée à l’utilisateur.

4.5 Patrons d’architecture mobilisés

Table 4.1 – Patrons d’architecture et de conception utilisés

Patron	Où	Pourquoi
Edge computing	Dispositif	Garantir le temps réel et le fonctionnement hors ligne
Publish/Subscribe	Broker Kafka	Diffuser la même image à toutes les versions simultanément
Scatter-Gather	Service de fusion	Regrouper les réponses des versions avec un délai maximal
API Gateway	Entrée du cloud	Point d’entrée unique, sécurité, limitation de débit
Circuit Breaker	Agent de synchronisation	Éviter les blocages en cas de panne réseau ou cloud
Strategy	Moteur de vote	Changer d’algorithme de fusion sans modifier le service
Database per service	Base des profils	Isoler les données sensibles et découpler les services

4.6 Sécurité et confidentialité

Les visages étant des données biométriques, plusieurs mesures sont prévues : chiffrement TLS de bout en bout, authentification des dispositifs et des objets connectés par certificats, transmission d’*embeddings* (vecteurs numériques résumant un visage) plutôt que de photos brutes, chiffrement de la base des profils et minimisation des données conservées, conformément au RGPD.

4.7 Choix technologiques

Table 4.2 – Technologies proposées par couche

Élément	Technologies
Dispositif	Android/Kotlin, TensorFlow Lite ou ONNX Runtime, modèles quantifiés
Objets connectés	Microcontrôleurs (Arduino, ESP32), MQTT [9], BLE
Edge local	Broker MQTT (EMQX ou Mosquitto), K3s
Ingestion	Envoy ou Kong, gRPC
Broker d'événements	Apache Kafka
Inférence	Python, NVIDIA Triton ou TorchServe, conteneurs Docker, GPU
Orchestration	Kubernetes (mise à l'échelle automatique)
Données	PostgreSQL avec l'extension pgvector
MLOps	MLflow
Supervision	Prometheus, Grafana

4.8 Justification : réponse aux exigences

Table 4.3 – Réponse de l'architecture proposée à chaque exigence

Exigence	Réponse apportée
E1 Temps réel	La décision est locale ; le cloud ne sert qu'à corriger, le temps réel ne dépend plus du réseau
E2 Flux	Un broker d'événements est conçu pour les flux continus, contrairement à REST
E3 Collectif	Le Publish/Subscribe distribue l'image à N versions, le Scatter-Gather regroupe les réponses
E4 Extensibilité	Une nouvelle version correspond à un conteneur supplémentaire, sans autre modification
E5 Hétérogénéité	Chaque couche utilise la technologie adaptée à son matériel
E6 Disponibilité	Mode dégradé local, persistance des messages dans Kafka, redémarrage automatique par Kubernetes
E7 Confidentialité	Chiffrement, embeddings au lieu de photos, base chiffrée et minimisation des données
E8 Scalabilité	Mise à l'échelle automatique par Kubernetes, qui remplace les formules fixes P_c^* et P_s^*
E9 Observabilité	Métriques de latence et de précision par version (Prometheus, Grafana)

4.9 Correspondance avec l’architecture initiale

Table 4.4 – Correspondance entre l’architecture initiale et l’architecture proposée

Élément de l’article	Dans la nouvelle architecture
Client lite (DNN// _c)	Dispositif edge avec modèles quantifiés et décision locale
Serveur multi-versions (DNN// _s)	Microservices d’inférence conteneurisés sur Kubernetes
Appels REST synchrones	Broker d’événements (Kafka, MQTT) et gRPC en streaming
MSParallelCollectiveManager	Service de fusion collective (Scatter-Gather, vote calibré)
ManagerServer et envoi d’alerte	Service d’alertes en mode push, uniquement en cas de désaccord
MSTinyParallelCollective	Objets connectés publiant sur MQTT, source optionnelle de la fusion
MSFaceDynamicTraining et redémarrage	Pipeline MLOps et registre de modèles, déploiement sans interruption
« Un service par processeur »	Orchestration et mise à l’échelle automatique par Kubernetes

4.10 Limites de la proposition

Aucune architecture n’est parfaite. Celle-ci est **plus complexe** à mettre en place et à exploiter : Kafka, Kubernetes et le MLOps exigent des compétences DevOps. Elle induit un **coût** d’infrastructure (GPU dans le cloud). Elle pose enfin un problème de **cohérence perçue** : l’utilisateur peut recevoir une annonce puis une correction, ce qui doit être géré du point de vue ergonomique, par exemple en n’annonçant localement que les décisions de forte confiance. Pour un prototype académique, il est possible de simplifier en remplaçant Kubernetes par Docker Compose et Kafka par un broker MQTT unique, tout en conservant la même logique.

Chapitre 5

Modélisation selon le modèle C4

5.1 Présentation du modèle C4

Le modèle C4, proposé par Simon Brown [3], décrit l'architecture d'un système logiciel à travers quatre niveaux de zoom successifs, à la manière d'une carte que l'on agrandit progressivement :

1. **Contexte** : le système vu comme une boîte, avec ses utilisateurs et les systèmes externes ;
2. **Conteneurs** : les applications et stockages déployables qui composent le système ;
3. **Composants** : l'organisation interne d'un conteneur ;
4. **Code** : les classes qui réalisent un composant (niveau facultatif).

Dans le modèle C4, un « conteneur » désigne une unité exécutable ou de stockage (application, service, base de données), et non nécessairement un conteneur Docker.

Table 5.1 – Notation utilisée dans les diagrammes C4

Élément	Représentation
Personne	Bloc bleu foncé surmonté d'une tête
Système logiciel étudié	Bloc bleu
Système externe	Bloc gris
Conteneur	Bloc bleu moyen ; bloc empilé lorsqu'il existe plusieurs instances
Base de données	Cylindre bleu moyen
Composant	Bloc bleu clair
Frontière	Rectangle en pointillés portant le nom du système ou du conteneur
Relation	Flèche en pointillés, annotée de l'action et du protocole entre crochets

Chaque élément indique son nom, entre crochets son type et sa technologie, puis une courte description de sa responsabilité.

5.2 Niveau 1 : diagramme de contexte

Le diagramme de contexte (figure 5.1) montre le système DNN// T_{cs} comme une boîte unique, entourée des personnes qui l'utilisent et des systèmes externes avec lesquels il interagit.

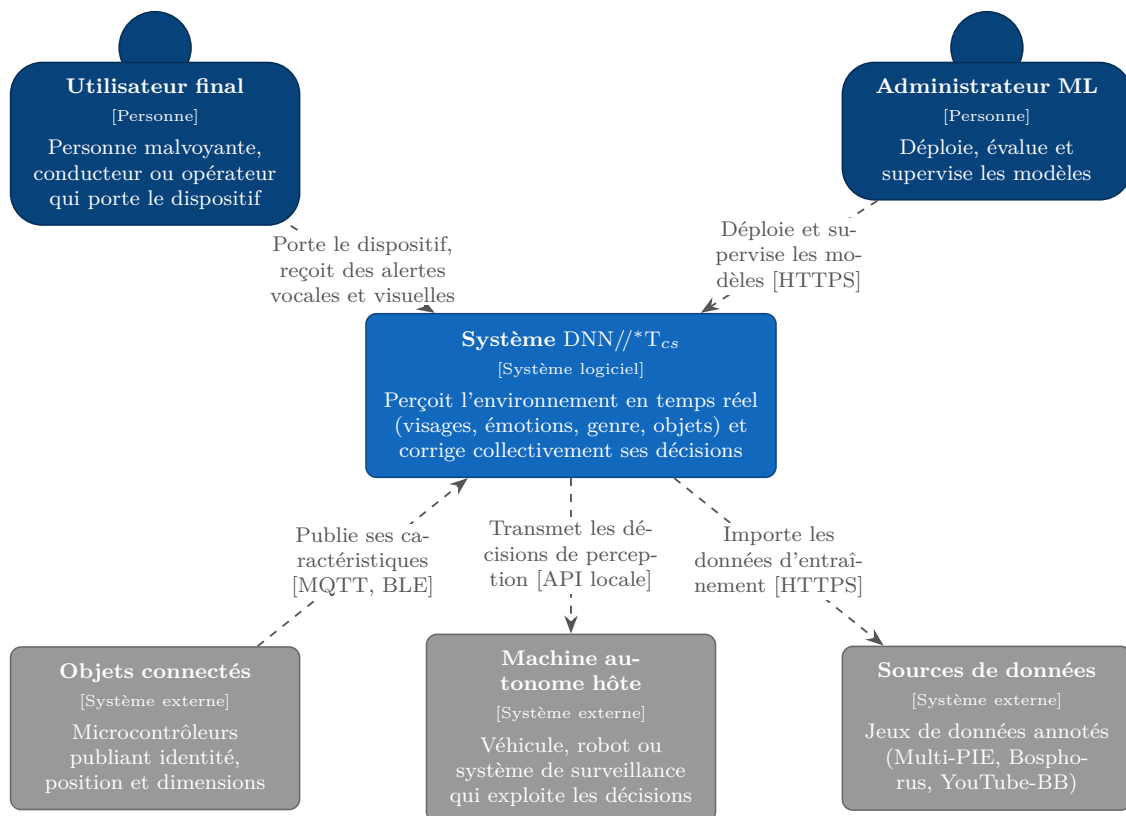


Figure 5.1 – C4 niveau 1 : diagramme de contexte du système

Lecture du diagramme. Deux types de personnes interagissent avec le système : l'**utilisateur final**, qui porte le dispositif et reçoit les alertes, et l'**administrateur ML**, qui gère les modèles. Trois systèmes externes gravitent autour : les **objets connectés**, qui fournissent des informations fiables sur eux-mêmes ; la **machine autonome hôte**, qui exploite les décisions (par exemple pour freiner) ; et les **sources de données**, utilisées pour entraîner et évaluer les modèles. À ce niveau, aucune technologie interne n'apparaît : le diagramme s'adresse à tous les publics, y compris non techniques.

5.3 Niveau 2 : diagramme de conteneurs

Le diagramme de conteneurs, présenté en pleine page à la figure 5.2, ouvre la boîte du système et montre ses unités déployables, leurs technologies et leurs échanges.

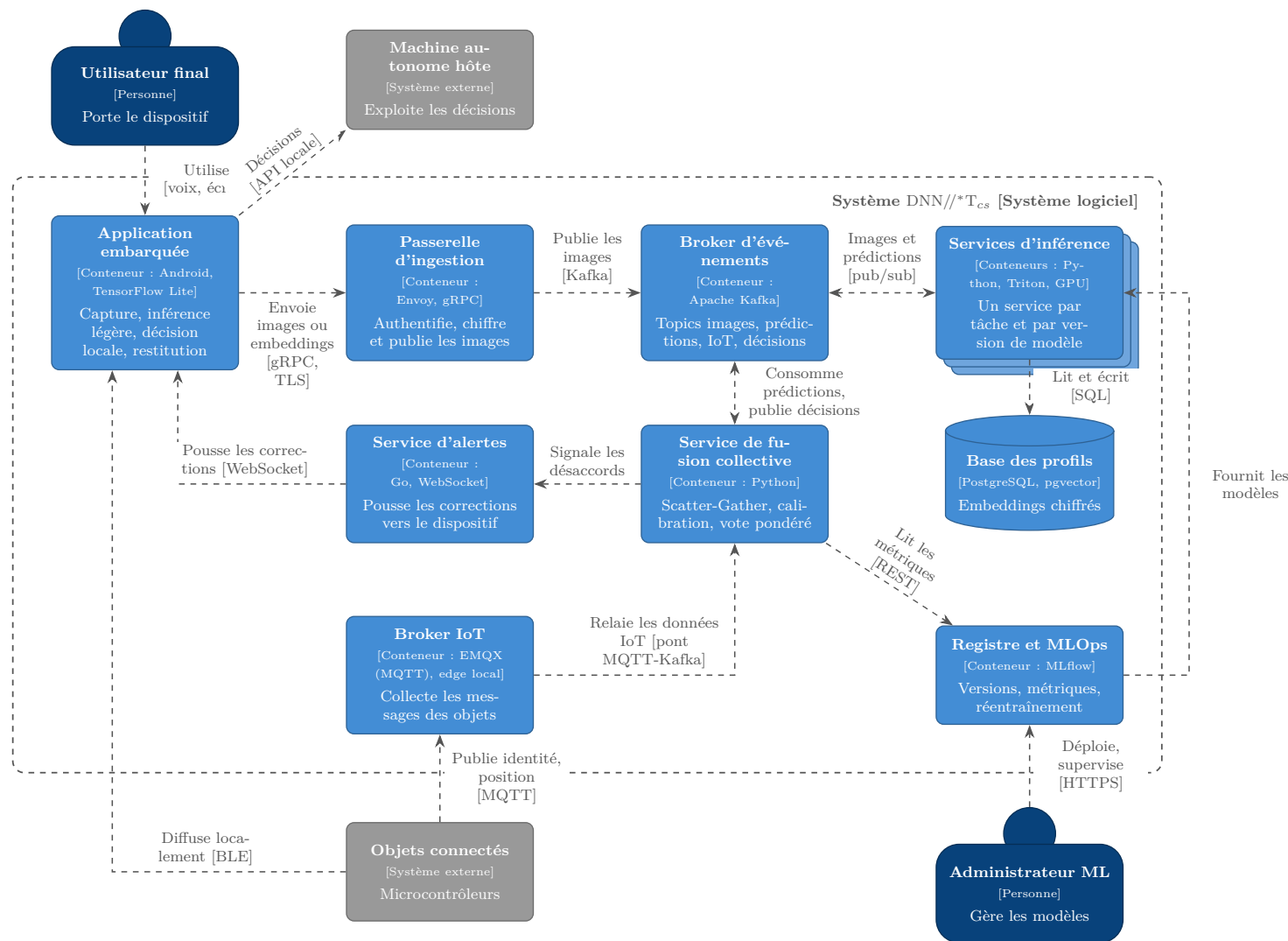


Figure 5.2 – C4 niveau 2 : diagramme de conteneurs

Lecture du diagramme. Le tableau 5.2 résume leurs responsabilités. Le flux principal suit la ligne supérieure : l'application embarquée prend une décision locale puis envoie les images à la passerelle, qui les publie dans Kafka ; les services d'inférence les consomment en parallèle et publient leurs prédictions ; le service de fusion les combine avec les données IoT et signale les désaccords au service d'alertes, qui pousse les corrections vers le dispositif.

Table 5.2 – Conteneurs du système et leurs responsabilités

Conteneur	Technologie	Responsabilité
Application embarquée	Android, TensorFlow Lite	Chemin critique local : capture, inférence légère, décision, restitution, synchronisation
Passerelle d'ingestion	Envoy, gRPC	Point d'entrée unique, authentification, chiffrement, publication des images
Broker d'événements	Apache Kafka	Distribution des événements entre services, persistance, découplage
Services d'inférence	Python, Triton, GPU	Une instance par tâche et par version de modèle ; mise à l'échelle indépendante
Service de fusion collective	Python	Regroupement, calibration et vote pondéré des prédictions
Service d'alertes	Go, WebSocket	Envoi des corrections au dispositif en cas de désaccord
Base des profils	PostgreSQL, pgvector	Stockage chiffré des embeddings de visages connus
Broker IoT	EMQX (MQTT)	Collecte des messages des objets connectés, au plus près du terrain
Registre et MLOps	MLflow	Versions des modèles, métriques de précision, réentraînement

5.4 Niveau 3 : diagramme de composants du service de fusion

Le niveau 3 zoome sur un conteneur précis. Nous avons choisi le service de fusion collective, car c'est lui qui réalise l'idée centrale de l'article ; son diagramme est présenté en pleine page à la figure 5.3.

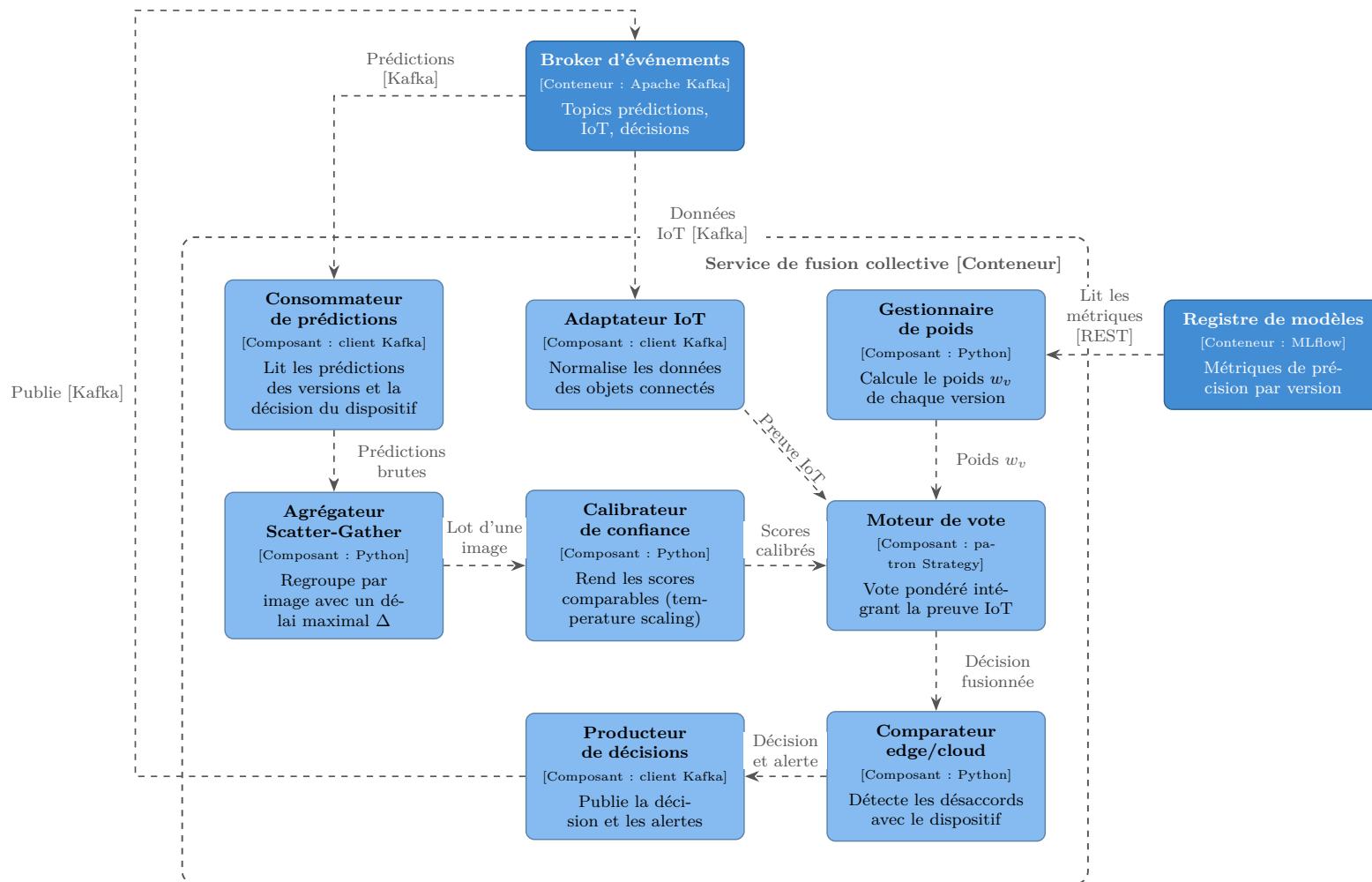


Figure 5.3 – C4 niveau 3 : composants du service de fusion collective

Lecture du diagramme. Le **consommateur de prédictions** lit les résultats de toutes les versions ainsi que la décision locale du dispositif. L'**agrégateur Scatter-Gather** les regroupe par identifiant d'image et attend au plus un délai Δ . Le **calibrateur** rend les scores comparables, puis le **moteur de vote** combine les prédictions en tenant compte des poids fournis par le **gestionnaire de poids** (calculés à partir des métriques du registre) et de la preuve fournie par l'**adaptateur IoT**. Le **comparateur** confronte la décision fusionnée à celle du dispositif et le **producteur** publie la décision, accompagnée d'une alerte en cas de désaccord.

5.5 Niveau 4 : code du moteur de fusion

Le niveau 4 est facultatif dans le modèle C4 ; il est présenté ici pour le cœur algorithmique du service de fusion, sous la forme d'un diagramme de classes UML (figure 5.4).

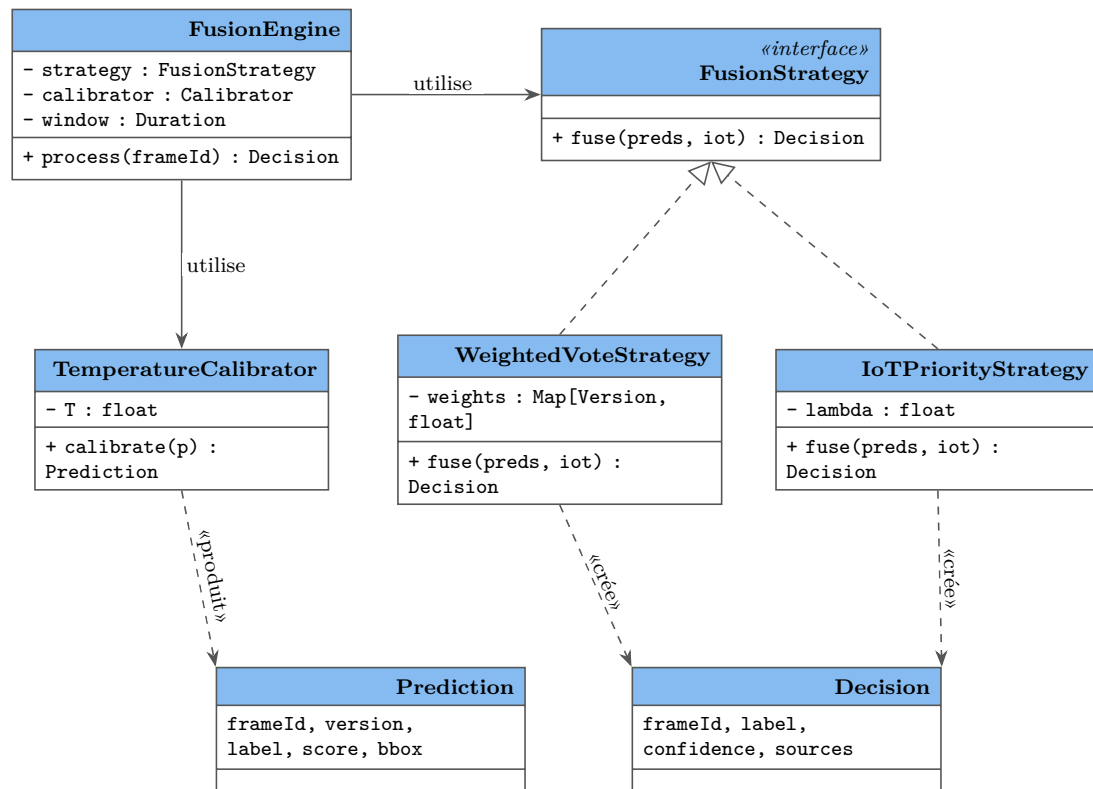


Figure 5.4 – C4 niveau 4 : diagramme de classes du moteur de fusion

La classe **FusionEngine** délègue la combinaison des prédictions à une interface **FusionStrategy** : c'est le patron **Strategy**. Deux stratégies sont proposées : **WeightedVoteStrategy**, qui réalise le vote pondéré de l'algorithme 1, et **IoTPriorityStrategy**, qui donne la priorité aux données des objets connectés lorsqu'elles sont disponibles. Une nouvelle méthode de fusion (par exemple un méta-classifieur appris) pourra être ajoutée sans modifier **FusionEngine**, conformément au principe ouvert/fermé. Le **TemperatureCalibrator** produit des objets **Prediction** dont les scores sont comparables entre versions.

5.6 Vue dynamique : traitement d'une image

Le modèle C4 propose, en complément, des diagrammes dynamiques qui montrent comment les éléments collaborent pour un scénario donné. Le tableau 5.3 décrit le traitement d'une image, en réutilisant les conteneurs du niveau 2.

Table 5.3 – Vue dynamique : traitement d’une image de bout en bout

N°	Source	Destination	Interaction
1	Application embarquée	Utilisateur final	Annonce la décision locale (environ 30 ms)
2	Application embarquée	Passerelle d’ingestion	Envoie l’image ou ses embeddings [gRPC, TLS]
3	Passerelle d’ingestion	Broker d’événements	Publie l’événement image
4	Broker d’événements	Services d’inférence	Diffuse l’image à toutes les versions en parallèle
5	Services d’inférence	Broker d’événements	Publient leurs prédictions
6	Broker IoT	Service de fusion	Relaie les données des objets présents
7	Service de fusion	Service d’alertes	Signale un désaccord avec la décision locale
8	Service d’alertes	Application embarquée	Pousse la correction [WebSocket]
9	Application embarquée	Utilisateur final	Annonce la décision corrigée

Conclusion générale

L'étude du système DNN// $\ast T_{cs}$ montre une idée pertinente : combiner l'exécution locale de modèles légers, la correction par des modèles plus lourds et l'apport d'objets connectés, afin de concilier temps réel et précision en vision embarquée.

L'analyse de son architecture a toutefois montré un écart entre l'intention et la réalisation. Il s'agit d'une architecture client-serveur à client lourd, étendue à trois tiers, dont les composants présentés comme des micro-services sont en réalité des processus d'une même application. Ce monolithe modulaire multi-processus souffre d'un couplage rigide au matériel, d'une communication synchrone mal adaptée aux flux vidéo, d'un point de défaillance unique et d'une absence de prise en compte de la sécurité.

Nous avons proposé une architecture hybride Edge-Fog-Cloud orientée événements. La décision immédiate est prise sur le dispositif, ce qui garantit le temps réel et le fonctionnement hors ligne. Dans le cloud, de véritables microservices, découplés par un broker d'événements, exécutent en parallèle toutes les versions des modèles ; un service de fusion fondé sur les patrons Publish/Subscribe et Scatter-Gather réalise la décision collective de manière calibrée, et les objets connectés deviennent une source d'information optionnelle. Cette architecture a été documentée selon les quatre niveaux du modèle C4.

Ses principales limites sont sa complexité opérationnelle et son coût. Une suite naturelle de ce travail serait la réalisation d'un prototype simplifié (Docker Compose, broker MQTT) permettant de mesurer effectivement la latence de bout en bout et le gain de précision apporté par la fusion collective.

Bibliographie

- [1] M. Helaoui et S. Bahroun, « Parallel Collective Tiny Deep Learning System for Face and Object Reidentification in Uncontrolled Environments », *Research Square* (prépublication), 2025. DOI : [10.21203/rs.3.rs-5108948/v1](https://doi.org/10.21203/rs.3.rs-5108948/v1).
- [2] S. Lee, D. Jha, A. Agrawal, A. Choudhary et W.-k. Liao, « Parallel deep convolutional neural network training by exploiting the overlapping of computation and communication », *IEEE 24th International Conference on High Performance Computing (HiPC)*, p. 183–192, 2017.
- [3] S. Brown, *The C4 model for visualising software architecture*, <https://c4model.com>.
- [4] J. Lewis et M. Fowler, « Microservices : a definition of this new architectural term », 2014, <https://martinfowler.com/articles/microservices.html>.
- [5] S. Newman, *Building Microservices*, 2^e édition, O'Reilly Media, 2021.
- [6] M. Richards et N. Ford, *Fundamentals of Software Architecture*, O'Reilly Media, 2020.
- [7] G. Hohpe et B. Woolf, *Enterprise Integration Patterns*, Addison-Wesley, 2003.
- [8] J. Kreps, N. Narkhede et J. Rao, « Kafka : a distributed messaging system for log processing », *Proceedings of the NetDB Workshop*, 2011.
- [9] OASIS, *MQTT Version 5.0*, OASIS Standard, 2019.
- [10] C. Guo, G. Pleiss, Y. Sun et K. Q. Weinberger, « On calibration of modern neural networks », *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [11] W. Shi, J. Cao, Q. Zhang, Y. Li et L. Xu, « Edge computing : vision and challenges », *IEEE Internet of Things Journal*, vol. 3, n° 5, p. 637–646, 2016.
- [12] F. Bonomi, R. Milito, J. Zhu et S. Addepalli, « Fog computing and its role in the internet of things », *Proceedings of the MCC Workshop on Mobile Cloud Computing*, p. 13–16, 2012.
- [13] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2^e édition, Pragmatic Bookshelf, 2018.